Semana 17 — Conteúdo de 2 aulas

Banco de Dados

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v9

Semana 17 — Conteúdo de 2 aulas

- **Aula 27:** Criando Tabelas: o Papel das Chaves Estrangeiras
- **Aula 28:** Criando Tabelas e Estabelecendo Relacionamentos

Aula 27 — Criando Tabelas: o Papel das Chaves Estrangeiras

Objetivos da Aula

- Definir uma **chave estrangeira** (FK).
- Entender a **integridade referencial**.
- Ligar tabelas na prática.

Relembrando

- A **FK** referencia a chave primária de outra tabela.
- Cria o **relacionamento**.
- Garante **integridade referencial**.

Definindo uma FK

```
CREATE TABLE Pedido (
    id INT PRIMARY KEY AUTO_INCREMENT,
    data DATE,
    id_cliente INT,
    FOREIGN KEY (id_cliente)
        REFERENCES Cliente(id)
);
```

O que a FK Garante

- Não se cria pedido de cliente **inexistente**.
- Não se apaga cliente com pedidos (por padrão).
- Mantém a **coerência**.

Ações Referenciais

- `ON DELETE CASCADE` — apaga os filhos.
- `ON DELETE SET NULL` — anula a referência.
- `ON DELETE RESTRICT` — bloqueia.
- Definem o comportamento ao excluir/atualizar.

Nomeando a FK

- Dar um **nome** ajuda a gerenciar.
- Ex.: `CONSTRAINT fk_pedido_cliente`.

Note

A **integridade referencial** é o que impede um pedido "órfão", apontando para um cliente que não existe.

Prática

- Crie `Cliente` e `Pedido`.
- Ligue `Pedido.id_cliente` a `Cliente.id`.
- Verifique a FK no `DESCRIBE`.

Resumo da Aula

- **FOREIGN KEY** cria a ligação entre tabelas.
- `REFERENCES` aponta para a chave primária.
- Ações referenciais definem o que fazer ao **excluir**.

Referências

- ROB, P.; CORONEL, C. *Sistemas de Banco de Dados*. Cengage.

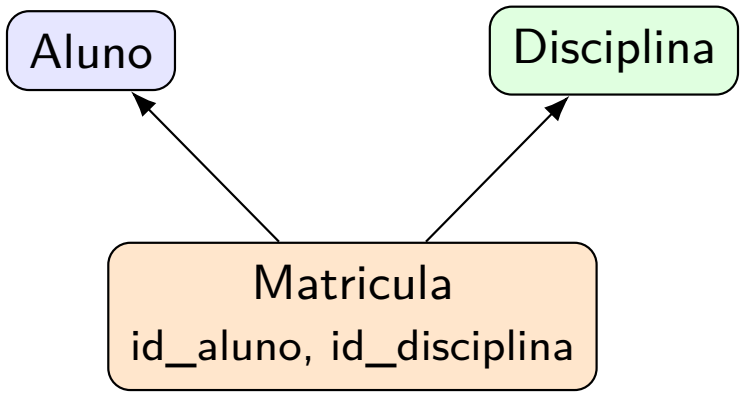
Aula 28 — Criando Tabelas e Estabelecendo Relacionamentos

Objetivos da Aula

- Criar um **conjunto** de tabelas relacionadas.
- Resolver relações **N:N** na prática.
- Consolidar o modelo **relacional**.

Do Modelo ER ao Banco

- Cada **entidade** vira uma tabela.
- Cada **relacionamento** vira uma **FK**.
- N:N vira uma **tabela intermediária**.



Ordem de Criação

- Primeiro as tabelas **sem FK**.
- Depois as que **dependem** delas.
- Evita erro de **referência inexistente**.

Exemplo — Escola

- **Curso** (id, nome).
- **Aluno** (id, nome, id_curso).
- **Disciplina** (id, nome).
- **Matricula** (id_aluno, id_disciplina).

Resolvendo N:N

- Aluno cursa várias disciplinas.
- Disciplina tem vários alunos.
- **Matricula** liga os dois → tabela intermediária.

```
CREATE TABLE Matricula (
    id_aluno INT,
    id_disciplina INT,
    PRIMARY KEY (id_aluno, id_disciplina),
    FOREIGN KEY (id_aluno) REFERENCES Aluno(id),
    FOREIGN KEY (id_disciplina) REFERENCES Disciplina(id)
);
```

i Note

O relacionamento N:N **nunca** fica "solto": ele sempre se materializa numa tabela com duas FKs.

Prática

- Implemente o modelo completo da escola.
- Confira as **relações** no Workbench.

Resumo da Aula

- Entidades viram **tabelas**; relações viram **FKs**.
- N:N vira **tabela intermediária**.
- A **ordem de criação** importa.

Referências

- HEUSER, C. A. *Projeto de Banco de Dados*. Bookman.